

Exercise 06: *PyUVM* Environment, Sequencer, and Sequences

Objective: Introduction to *PyUVM* environments, sequencers and sequences.

Task: Expanding the existing *PyUVM* tests for the Saturation Filter filter with a *PyUVM* environment virtual sequencer and virtual sequences.

In this exercise you need to extend your existing *PyUVM* tests for the *Saturation Filter* filter from *Exercise 05: PyUVM Test* with a *PyUVM* environment, virtual sequencer, and virtual sequences.

This exercise introduces a number of new classes and storing all of them in a single source file would make the code illegible, therefore from now on we will try to limit the number of class definitions per source file to one.

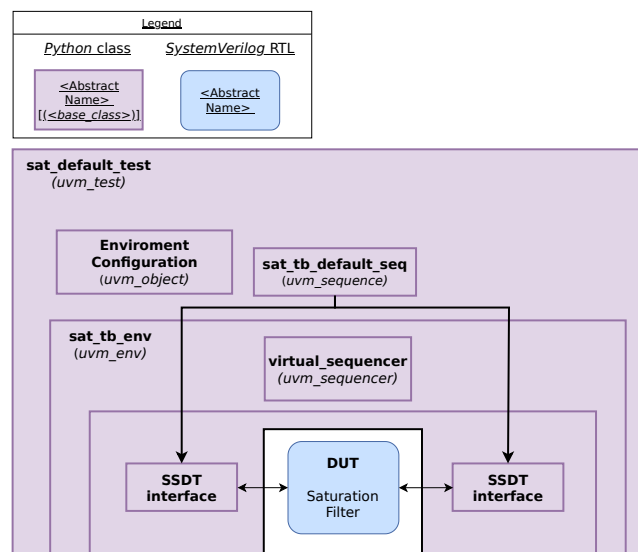


Fig. 0.1: Testbench overview. This exercise implements this class structure for the *Saturation Filter* *PyUVM* testbench.

Part 1: *PyUVM* Simple test

<ROOT>/lecture04/exercises/E06_sat_pyuvmm_env_vseq contains the Makefile and the base files for all necessary components.

Virtual Sequencer and Environment

First the virtual sequencer and environment classes need to be created.

In `sat_tb_virt_sequencer.py` create a virtual sequencer class, extending `uvm_sequencer`.

- Overwrite the `__init__` to set a default name for the class and create an empty handle:
 - `cfg` the configuration object.
 - Don't forget to call `super().__init__()`

In `sat_tb_env.py` create a new environment class extending `uvm_env`.

- Overwrite the `__init__` to set a default name for the class and create empty handles:

- `cfg`, to store the configuration object.
- `virtual_sequencer` for the virtual sequencer.
- In the `build_phase` create a virtual sequencer using the factory.

In `sat_defalut_test.py` create a *PyUVM* test. You can refactor most your interface test from Part 3 of Exercise 05.

- The clock setup and DUT reset remain in the `body` default test.
- Remove the stimuli generation and checking, we will need it when creating the virtual sequence.
- In the `build_phase` create the environment using the factory.

Testbench configuration

Next we need to create is the testbench configuration. The config typically contains parameters to adapt the testbench to changing DUT parameters. It is also used to share the DUT interface classes. This is necessary, since the stimuli generation will be moved into a sequence and the sequences does not have direct access to the interface.

Create a testbench config object in `sat_tb_config.py`, extending `uvm_object`. The config shall have two empty handles for the input and output interface:

- Overwrite the `__init__` to set a default name for the class and create empty handles:
 - `input_if`, to store the input interface class handle.
 - `output_if`, to store the output interface class handle.

In the `build_phase` of the default test, after instantiating the interfaces, create a config object using the factory. Populate the configuration with the interface handles.

We will need access the configuration object in the environment and the virtual sequencer. To share objects between parts of the testbench use the `ConfigDB`.

Variables are *set* and retrieved (*get*) form the `ConfigDB` using the `ConfigDB().get()` and `ConfigDB().set()` methods:

ConfigDB set()

```
def set(context, inst_name, field_name, value)
```

Stores an object in the db using the context and `inst_name` to create a retrieval path, and the key name.

Arguments:

- `context`: A handle to a component
- `inst_name`: The instance name within the component
- `field_name`: The key we're setting
- `value`: The object to be stored
- Returns: None

ConfigDB get()

```
def get(context, inst_name, field_name, default=default_get)
```

The component path matches against the paths in the ConfigDB. The path cannot have wildcards, but can match against keys with wildcards. Return the value stored at key.

Arguments:

- **context**: The component making the call
- **inst_name**: component full path with no wildcards
- **field_name**: the **field_name** being retrieved
- **default**: the value to return if there is no key
- *returns*: value found at location

- In the **build_phase** of the default test, set the **cfg** field of the environment instance to the handle of the configuration object.
- In the **build_phase** of the environment, retrieve the configuration object from the ConfigDB
- Then set the **cfg** field for the created virtual sequencer to the configuration object handle.
- In the **build_phase** of the virtual sequencer, retrieve the configuration object again.

Top sequence

Lastly we need the sequence to drive the stimuli. In **sat_tb_default_seq** create a sequence class extending **uvm_sequence**.

- Overwrite the **__init__** to set a default name for the class and create empty handles:
 - **sequencer**, for the handle to the sequencer.
 - **cfg**, for the handle to the configuration object.
- Every **uvm_sequencer** automatically has a handle to the sequencer running the sequence.¹
- In the **pre_body** populate the **cfg** handle with the configuration object from the sequencer. **self.cfg = self.sequencer.cfg**
- Optionally the interface handles can also be assigned to local instance variables for easier use.
- In the **body** refactor your stimuli generation from Exercise 05.

Create a new default sequence in the **run_phase** of the default test and start the sequence on the virtual sequencer using **<sequence>.start(seqr=<sequencer_handle>)**. You can access the virtual sequencer through the environment handle.

Part 2: Random Test

Refactor the Random Test from Exercise 05 to use the environment and a **uvm_sequence**.

- Create a **uvm_sequence** in **sat_tb_random_seq.py** extending the default sequence.

¹You can inspect the sourcecode **uvm_sequence.start()** to see how the sequence handle is propagated

- Overwrite the `body` with the random stimuli generation.
- Create a new test in `sat_random_test.py` extending the default test.
- In the `start_of_simulation_phase` create a factory override by type to replace the default sequence with the random sequence.

Factory type override by type

```
def set_type_override_by_type(original_type, override_type, replace=True)
```

Override one type with another type globally

- `original_type`: The original type to be overridden
- `override_type`: The new type that will override it
- `replace`: If the override exists, only replace it if this is `True`
- returns: `None`